

Equipment Management System

프로젝트 보고서

1. 프로젝트 개요

프로젝트명

Equipment Management System

프로젝트 목적

제조 현장에서 사용되는 장비 정보를 효율적으로 관리하기 위한 웹 기반 장비 관리 시스템을 구현했습니다.

Spring Boot를 이용하여 장비 관리 REST API를 구축하고, React를 이용하여 사용자가 장비 정보를 직관적으로 조회하고 관리할 수 있는 프론트엔드 화면을 구현했습니다.

단순한 CRUD 구현에 그치지 않고 데이터베이스 연동, 입력값 검증, 예외 처리, 검색 및 필터링, 대시보드, API 문서화, 테스트 등 실제 웹 서비스 개발 과정에서 필요한 기능을 직접 구현하는 것을 목표로 진행했습니다.

프로젝트 저장소

<https://github.com/zuzusnasna/equipment-management>

2. 프로젝트 개발 목표

3. Spring Boot 기반 REST API 설계 및 구현

4. Spring Data JPA를 이용한 데이터베이스 연동

5. 장비 정보 CRUD 기능 구현

6. 장비 상태 및 이름 기반 검색 기능 구현

7. 장비 정보를 효과적으로 확인할 수 있는 대시보드 구현

8. React 기반 프론트엔드 구현

9. REST API와 React 프론트엔드 연동

10. Jakarta Validation을 이용한 입력값 검증

11. 전역 예외 처리 구현

12. JUnit과 Mockito를 활용한 테스트

13. Swagger/OpenAPI를 활용한 API 문서화

14. Git과 GitHub를 이용한 버전 관리

15. 개발 환경

Backend

Java 26

Spring Boot 4.1.0

Spring Data JPA

Spring Web

Jakarta Validation

Oracle Database

Maven

Swagger / OpenAPI

JUnit

Mockito

Frontend

JavaScript

React

Vite

Fetch API

CSS

개발 도구

IntelliJ IDEA

Visual Studio Code

Git

GitHub

4. 프로젝트 전체 구조

프로젝트는 크게 Spring Boot 기반의 Backend와 React 기반의 Frontend로 구성했습니다.

프로젝트 구조

equipment-management

|—— src

```
|   |─── main
|   |   |─── java
|   |   |   |─── com.example.equipmentmanagement
|   |   |   |   |─── equipment
|   |   |   |   |   |─── Equipment.java
|   |   |   |   |   |─── EquipmentController.java
|   |   |   |   |   |─── EquipmentRepository.java
|   |   |   |   |   |─── EquipmentService.java
|   |   |   |   |   |─── EquipmentNotFoundException.java
|   |   |   |   |   |─── GlobalExceptionHandler.java
|   |   |   |
|   |   |   |─── dto
|   |   |   |   |─── EquipmentRequest.java
|   |   |   |   |─── EquipmentResponse.java
|   |   |
|   |   |─── resources
|   |   |   |─── application.properties
|   |
|   |─── test
|   |   |─── java
|   |   |   |─── com.example.equipmentmanagement
|   |   |   |   |─── equipment
|   |   |   |   |   |─── EquipmentControllerTest.java
|   |   |   |   |   |─── EquipmentServiceTest.java
|   |
|─── frontend
|   |─── src
|   |   |─── api
|   |   |   |─── equipmentApi.js
|   |   |─── App.jsx
|   |   |─── App.css
|   |   |─── main.jsx
|
|─── sql
|─── pom.xml
```

└── README.md

5. 시스템 구성

전체적인 데이터 흐름은 다음과 같이 구성했습니다.

사용자

→ React 화면

→ Fetch API

→ Spring Boot REST Controller

→ Service

→ Repository

→ Oracle Database

조회 요청의 경우 데이터베이스에서 조회한 Entity를 Service를 통해 전달하고, Controller에서 Response DTO로 변환하여 JSON 형태로 프론트엔드에 반환하도록 구성했습니다.

등록이나 수정 요청에서는 React에서 입력한 데이터를 Request DTO로 전달하고 Validation을 적용한 후 Service 계층에서 실제 데이터 처리를 수행하도록 구성했습니다.

6. Backend 구현

6-1. Controller

EquipmentController는 장비와 관련된 REST API 요청을 처리하는 역할을 담당합니다.

기본 URL은 /equipments로 구성했습니다.

구현한 API

GET /equipments

전체 장비 목록을 조회합니다.

GET /equipments/{id}

특정 장비의 상세 정보를 조회합니다.

POST /equipments

새로운 장비를 등록합니다.

PUT /equipments/{id}

기존 장비 정보를 수정합니다.

DELETE /equipments/{id}

특정 장비를 삭제합니다.

GET /equipments/search?status={status}

장비 상태를 기준으로 검색합니다.

GET /equipments/search/name?name={name}

장비 이름을 기준으로 검색합니다.

Controller에는 Swagger/OpenAPI 애너테이션을 적용하여 각 API의 기능과 응답 상태를 문서화했습니다.

또한 POST와 PUT 요청에는 @Valid를 적용하여 Request DTO의 입력값 검증이 수행되도록 구성했습니다.

6-2. Service

EquipmentService는 실제 장비 관리 비즈니스 로직을 담당합니다.

주요 기능

전체 장비 조회

장비 단건 조회

장비 등록

장비 수정

장비 삭제

상태별 장비 조회

장비 이름 검색

Controller에서 전달받은 요청을 직접 데이터베이스에 처리하지 않고 Service 계층을 거치도록 구성하여 Controller와 데이터 접근 로직을 분리했습니다.

장비를 조회하거나 수정·삭제하는 과정에서 존재하지 않는 ID가 전달되는 경우 EquipmentNotFoundException을 발생시키도록 구현했습니다.

6-3. Repository

EquipmentRepository는 Spring Data JPA의 JpaRepository를 상속하여 기본적인 CRUD 기능을 사용할 수 있도록 구성했습니다.

추가적으로 장비 상태 검색과 장비 이름 검색을 위해 Spring Data JPA의 메서드 이름 기반 쿼리 기능을 활용했습니다.

`findByStatus(String status)`

장비 상태를 기준으로 검색합니다.

`findByNameContaining(String name)`

장비 이름에 특정 문자열이 포함되어 있는 장비를 검색합니다.

별도의 SQL을 직접 작성하지 않고 Repository 메서드 이름만으로 검색 조건을 표현하여 데이터 접근 코드를 단순화했습니다.

7. DTO를 이용한 데이터 전달

Entity 객체를 Controller에서 직접 사용하는 대신 Request DTO와 Response DTO를 분리하여 사용했습니다.

EquipmentRequest

장비 등록 및 수정 요청에 사용합니다.

EquipmentResponse

장비 조회 결과를 클라이언트에 전달할 때 사용합니다.

이를 통해 외부에서 전달되는 요청 데이터와 실제 데이터베이스 Entity를 분리하고 API 응답 구조를 관리할 수 있도록 구성했습니다.

8. 장비 CRUD 기능

장비 관리 시스템의 핵심 기능으로 CRUD를 구현했습니다.

Create

사용자가 장비명, 장비 유형, 상태, 위치를 입력하여 새로운 장비를 등록합니다.

Read

전체 장비 목록을 조회하거나 장비 ID를 이용하여 특정 장비의 상세 정보를 조회할 수 있습니다.

Update

장비 ID를 기준으로 기존 장비 정보를 조회한 후 장비명, 유형, 상태, 위치를 수정합니다.

Delete

장비 ID를 기준으로 장비를 조회하고 해당 장비를 삭제합니다.

이를 통해 실제 장비 관리 시스템에서 기본적으로 필요한 데이터 관리 기능을 구현했습니다.

9. 검색 및 필터링 기능

등록된 장비를 효율적으로 찾을 수 있도록 검색 기능을 추가했습니다.

상태 검색

장비 상태를 기준으로 장비를 검색할 수 있습니다.

예를 들어 정상, 점검, 고장 등의 상태별로 장비 목록을 조회할 수 있도록 구성했습니다.

이름 검색

장비 이름에 포함된 문자열을 기준으로 검색할 수 있도록 구현했습니다.

예를 들어 "Bonder"라는 검색어를 입력하면 해당 문자열이 포함된 장비를 조회할 수 있습니다.

이 기능을 통해 장비 수가 많아졌을 때 원하는 장비를 빠르게 찾을 수 있도록 했습니다.

10. 입력값 검증

장비 등록 및 수정 과정에서 잘못된 데이터가 저장되지 않도록 Jakarta Validation을 적용했습니다.

Controller에서 @Valid를 사용하여 Request DTO의 검증 조건을 적용하도록 구성했습니다.

검증 과정에서 문제가 발생하면 GlobalExceptionHandler가 해당 예외를 처리하여 클라이언트에 오류 정보를 반환합니다.

이를 통해 잘못된 요청에 대한 일관된 처리가 가능하도록 구현했습니다.

11. 전역 예외 처리

API에서 발생하는 예외를 각각의 Controller에서 처리하지 않고 GlobalExceptionHandler를 통해 공통적으로 처리하도록 구성했습니다.

처리하는 주요 예외

EquipmentNotFoundException

존재하지 않

1. 프로젝트 개요

1.1 프로젝트명

GAMEHUB

1.2 프로젝트 유형

Java Servlet 기반 게임 커뮤니티 웹 애플리케이션

1.3 GitHub 저장소

<https://github.com/zuzusnasna/miniproject>

1.4 프로젝트 소개

GAMEHUB는 게임 카테고리별 게시판을 중심으로 회원들이 게시글과 댓글을 작성하고 서로 소통할 수 있도록 구현한 게임 커뮤니티 웹 애플리케이션이다.

단순한 게시판 CRUD 구현에 그치지 않고 회원 등급과 역할(Role)을 기반으로 권한을 관리하고, 게임 카테고리별 관리자 지정 및 관리자 신청 기능까지 구현하여 실제 커뮤니티 서비스와 유사한 구조를 목표로 개발하였다.

회원, 게시글, 댓글, 좋아요 및 싫어요, 게임 카테고리, 카테고리 관리자 등의 기능을 관계형 데이터베이스와 연결하여 하나의 웹 애플리케이션으로 구성하였다.

2. 프로젝트 개발 목적

본 프로젝트의 주요 목적은 Java Servlet 기반 웹 애플리케이션의 전체적인 구조를 직접 구현하고 이해하는 것이었다.

특히 다음과 같은 목표를 가지고 개발하였다.

- Java Servlet 기반 웹 애플리케이션 구현
- MVC 형태의 계층 구조 이해
- Servlet과 DAO/DTO의 역할 분리
- Oracle Database 연동
- JDBC를 이용한 데이터 처리
- 회원 및 게시글 CRUD 구현
- 게임 카테고리 기반 게시판 구현
- 회원 등급 및 역할 기반 권한 관리
- 카테고리 관리자 신청 및 관리 기능 구현
- JavaScript Fetch API를 이용한 비동기 데이터 처리
- Git을 활용한 기능 단위 개발 및 협업 경험

3. 개발 환경

구분	기술
Language	Java
Backend	Servlet / Jakarta Servlet
Server	Apache Tomcat 10.1
Database	Oracle Database
DB Access	JDBC
Frontend	HTML5 / CSS3 / JavaScript
Data Format	JSON
Build	Maven
IDE	IntelliJ IDEA
Version Control Git / GitHub	

4. 시스템 구성

GAMEHUB는 크게 화면 영역, Servlet 영역, DAO 영역, DTO 영역, 데이터베이스 영역으로 구성하였다.

사용자가 웹 브라우저에서 요청을 보내면 Servlet이 요청을 처리하고 DAO를 통해 Oracle Database에 접근한다.

조회된 데이터는 DTO 객체로 전달하고 Servlet에서 JSON 또는 화면 데이터 형태로 변환하여 클라이언트에 전달하는 구조로 구현하였다.

전체적인 구조는 다음과 같다.

사용자

↓

HTML / CSS / JavaScript

↓

Servlet

↓

DAO

↓

JDBC

↓

Oracle Database

5. 프로젝트 아키텍처

5.1 Servlet Layer

Servlet Layer에서는 클라이언트의 HTTP 요청을 받아 실제 기능을 처리한다.

주요 Servlet은 다음과 같다.

- LoginServlet
- LogoutServlet
- SignupServlet
- MemberUpdateServlet
- FindIdServlet
- FindPasswordServlet
- PostListServlet
- PostDetailServlet
- PostWriteServlet
- PostEditServlet
- PostUpdateServlet
- PostDeleteServlet
- PostLikeServlet
- CommentLikeServlet

- CategoryServlet
- CategoryManagerRequestServlet
- AdminCategoryManagerRequestServlet
- GameNewsServlet

각 Servlet은 특정 기능을 담당하도록 분리하여 관리하였다.

5.2 DAO Layer

DAO(Data Access Object)는 데이터베이스와 직접 연결되어 SQL을 실행하는 역할을 담당한다.

주요 DAO는 다음과 같다.

- MemberDAO
- PostDAO
- CategoryDAO
- CategoryManagerRequestDAO
- Comment 관련 DAO
- Like 관련 DAO

Servlet에서 SQL을 직접 실행하지 않고 DAO 계층으로 데이터베이스 접근을 분리함으로써 각 계층의 책임을 구분하였다.

5.3 DTO Layer

DTO(Data Transfer Object)는 데이터베이스에서 조회하거나 애플리케이션에서 전달해야 하는 데이터를 객체 형태로 관리하기 위해 사용하였다.

주요 DTO는 다음과 같다.

- MemberDTO
- PostDTO
- CategoryDTO

- 댓글 관련 DTO
- 좋아요 관련 DTO
- 관리자 관련 DTO

DTO를 활용하여 데이터 전달 구조를 명확하게 구성하였다.

6. 주요 기능

6.1 회원 기능

GAMEHUB에서는 기본적인 회원 관리 기능을 구현하였다.

주요 기능은 다음과 같다.

- 회원가입
- 로그인
- 로그아웃
- 마이페이지
- 회원 정보 조회
- 회원 정보 수정
- 아이디 찾기
- 비밀번호 찾기
- 회원 레벨 관리
- 계정 상태 관리
- 좋아요/싫어요 활동 정보

회원의 가입 상태와 계정 상태를 별도로 관리하여 서비스 이용 상태를 구분할 수 있도록 설계하였다.

7. 게임 카테고리 기능

게임별 커뮤니티를 제공하기 위해 계층형 카테고리 구조를 구현하였다.

주요 기능은 다음과 같다.

- 게임 카테고리 조회
- 계층형 카테고리 구성
- 카테고리별 게시판 이동
- 카테고리 활성화/비활성 관리
- 카테고리 관리자 기능

카테고리는 부모-자식 관계를 가지도록 설계하여 게임 카테고리 하위 게시판을 계층적으로 관리할 수 있도록 구성하였다.

이를 통해 하나의 게시판이 아니라 여러 게임을 하나의 커뮤니티 서비스에서 관리할 수 있도록 구현하였다.

8. 게시글 기능

게시판의 핵심 기능으로 게시글 CRUD를 구현하였다.

주요 기능은 다음과 같다.

- 게시글 작성
- 게시글 목록 조회
- 게시글 상세 조회
- 게시글 수정
- 게시글 삭제
- 조회수 증가
- 공지사항 구분
- 좋아요
- 싫어요
- 작성자 기반 수정 및 삭제 권한 처리

게시글에는 작성자와 카테고리 정보를 연결하여 어떤 사용자가 어떤 게임 카테고리에 작성한 게시글인지 확인할 수 있도록 구성하였다.

또한 게시글 삭제 여부를 별도의 상태값으로 관리하여 데이터 처리 과정에서 삭제 상태를 구분할 수 있도록 설계하였다.

9. 댓글 기능

게시글에 대한 사용자 간 소통을 위해 댓글 기능을 구현하였다.

주요 기능은 다음과 같다.

- 댓글 작성
- 댓글 조회
- 댓글 삭제
- 대댓글
- 댓글 좋아요
- 댓글 싫어요

댓글은 자기 자신을 참조할 수 있는 구조를 이용하여 대댓글을 구현하였다.

이를 통해 하나의 게시글에 여러 댓글이 연결되고, 댓글에 다시 답글을 작성할 수 있는 계층적인 댓글 구조를 구현하였다.

10. 좋아요 / 싫어요 기능

게시글과 댓글에 대해 좋아요 및 싫어요 기능을 구현하였다.

게시글 좋아요와 댓글 좋아요를 별도의 데이터로 관리하여 사용자의 반응을 저장할 수 있도록 구성하였다.

또한 동일한 사용자가 동일한 게시글 또는 댓글에 중복으로 반응하지 않도록 회원과 게시글 또는 댓글을 조합한 구조로 데이터를 관리하였다.

이를 통해 단순한 숫자 증가 방식이 아니라 사용자별 반응을 추적할 수 있도록 구현하였다.

11. 회원 등급 및 권한 관리

GAMEHUB의 주요 특징 중 하나는 회원 등급과 역할을 이용한 권한 관리이다.

회원에게 역할(Role)을 부여하고 역할에 따라 사용할 수 있는 기능을 구분하였다.

주요 권한 구조는 다음과 같다.

- MEMBER
- ROLE 기반 권한
- 카테고리 관리자
- 시스템 관리자 기능

일반 회원과 관리자 기능을 분리하고, 특정 카테고리를 관리할 수 있는 사용자를 별도로 지정할 수 있도록 구현하였다.

12. 카테고리 관리자 기능

게임 카테고리별로 관리자를 지정할 수 있는 기능을 구현하였다.

사용자는 특정 카테고리의 관리자를 신청할 수 있으며 관리자는 신청 내역을 확인하고 승인 또는 거절할 수 있도록 구성하였다.

주요 기능은 다음과 같다.

- 카테고리 관리자 신청
- 관리자 신청 상태 관리
- 관리자 승인
- 관리자 거절
- 카테고리 관리자 지정
- 카테고리별 관리 권한 관리

이를 통해 전체 시스템을 관리하는 관리자와 특정 게임 게시판만 관리하는 카테고리 관리자의 역할을 구분하였다.

13. 카테고리 이용 제한

특정 회원의 카테고리 이용을 제한할 수 있는 기능도 설계하였다.

카테고리 이용 제한 정보를 별도로 관리하고 제한 대상 회원과 제한을 처리한 관리자를

구분하여 저장하도록 구성하였다.

이를 통해 단순히 회원 전체의 활동을 제한하는 것이 아니라 특정 게임 카테고리에서만 이용을 제한할 수 있는 구조를 구현하였다.

14. 관리자 신청 기능

사용자가 카테고리 관리자가 되기 위해 신청할 수 있는 기능을 구현하였다.

관리자 신청 데이터에는 신청 회원, 신청 카테고리, 신청 사유, 신청 상태, 신청 일시 및 처리 일시 등의 정보를 관리하도록 구성하였다.

관리자는 신청 내용을 확인한 후 승인 또는 거절할 수 있으며, 승인된 사용자는 해당 카테고리를 관리할 수 있도록 설계하였다.

이를 통해 단순히 관리자가 직접 권한을 부여하는 구조가 아니라 신청 → 검토 → 승인이라는 실제 서비스와 유사한 흐름을 구현하였다.

15. 데이터베이스 설계

Oracle Database를 사용하여 회원, 역할, 카테고리, 게시글, 댓글, 좋아요 및 관리자 관련 데이터를 관계형 구조로 관리하였다.

주요 테이블은 다음과 같다.

- MEMBER
- ROLE
- MEMBER_ROLE
- CATEGORY
- CATEGORY_MANAGER
- CATEGORY_MANAGER_REQUEST
- CATEGORY_BAN
- MANAGER_REQUEST
- POST
- POST_COMMENT

- POST_LIKE
- COMMENT_LIKE

회원과 역할은 MEMBER_ROLE을 통해 연결하고, 회원과 게시글은 MEMBER_NO를 통해 연결하였다.

게시글과 댓글은 POST_ID를 통해 연결하며 댓글은 PARENT_COMMENT_ID를 이용하여 대댓글 구조를 구현하였다.

게시글 및 댓글의 좋아요 정보는 각각 POST_LIKE와 COMMENT_LIKE 테이블을 통해 관리하였다.

16. 데이터베이스 관계 구조

회원과 역할은 다대다 관계를 MEMBER_ROLE을 통해 관리하였다.

회원은 여러 게시글을 작성할 수 있으며 하나의 게시글은 하나의 작성자를 가진다.

하나의 게시글은 여러 댓글을 가질 수 있으며 댓글 작성자 역시 회원과 연결된다.

댓글은 자기 자신을 참조할 수 있도록 구성하여 대댓글 관계를 표현하였다.

카테고리는 부모 카테고리를 참조하는 구조를 통해 계층형 카테고리를 구성하였다.

이러한 관계를 통해 GAMEHUB의 회원, 게시글, 댓글, 카테고리 및 권한 데이터를 하나의 관계형 구조로 연결하였다.

17. 프론트엔드와 백엔드 연결

HTML, CSS, JavaScript를 이용하여 사용자 화면을 구현하였다.

JavaScript에서는 Fetch API를 활용하여 Servlet과 통신하고 JSON 데이터를 받아 화면에 반영하는 방식을 사용하였다.

전체적인 데이터 흐름은 다음과 같다.

사용자 화면

↓

JavaScript

↓

Fetch API

↓

Servlet

↓

DAO

↓

Oracle Database

↓

DAO

↓

Servlet

↓

JSON

↓

JavaScript

↓

화면 출력

이를 통해 서버에서 데이터를 조회하고 이를 클라이언트 화면에 동적으로 반영하는 과정을 경험하였다.

18. 프로젝트에서 경험한 트러블슈팅

18.1 Servlet과 세션 사용자 정보 불일치

로그인 이후 세션에 저장된 사용자 정보와 다른 Servlet에서 참조하는 세션 속성명이 일치하지 않는 문제를 확인하였다.

로그인 Servlet에서 저장한 사용자 정보를 기준으로 다른 Servlet에서도 동일한 세션 데이터를 사용하도록 흐름을 확인하고 수정하였다.

이를 통해 웹 애플리케이션에서 로그인 상태를 유지하기 위해 세션 데이터가 여러 기능 사이에서 일관되게 관리되어야 한다는 점을 학습하였다.

18.2 게시물 권한 처리 문제

게시글 수정 및 삭제 기능에서 단순히 로그인 여부만 확인하는 것이 아니라 게시물 작성자와 현재 로그인 사용자를 비교해야 했다.

또한 일반 사용자와 카테고리 관리자, 시스템 관리자에 따라 삭제할 수 있는 게시물 범위가 달라지도록 권한 구조를 고려하였다.

이를 통해 기능 구현 과정에서 인증과 권한 검사를 함께 고려해야 한다는 점을 경험하였다.

18.3 카테고리 관리자 권한 문제

카테고리 관리자 기능을 구현하면서 특정 관리자가 담당 카테고리에서만 관리 기능을 수행할 수 있도록 권한 범위를 확인하였다.

관리자 신청과 승인 과정에서 회원, 카테고리, 역할 데이터가 서로 연결되기 때문에 각 DAO와 Servlet의 데이터 흐름을 확인하면서 문제를 해결하였다.

18.4 게시물 카테고리 검증

게시글 작성 및 수정 과정에서 전달된 카테고리 ID가 실제 존재하는 카테고리인지 확인할 필요가 있었다.

CategoryDAO를 통해 카테고리 정보를 검증하는 구조로 개선하여 잘못된 카테고리 데이터가 게시물에 연결되는 것을 방지하였다.

18.5 Servlet 코드 리팩터링

프로젝트 후반에는 여러 Servlet의 코드를 다시 검토하고 구조를 정리하였다.

PostListServlet, PostLikeServlet, CategoryServlet, CategoryManagerRequestServlet, AdminCategoryManagerRequestServlet, FindIdServlet, FindPasswordServlet, MemberUpdateServlet, PostEditServlet, PostUpdateServlet, SignupServlet 등의 코드를 리

팩터링하였다.

불필요한 디버깅 코드를 제거하고 코드 구조를 정리하여 유지보수하기 쉬운 형태로 개선하였다.

19. Git 협업 및 형상관리

프로젝트에서는 Git과 GitHub를 이용하여 기능 단위로 개발 내용을 관리하였다.

기본적인 개발 흐름은 다음과 같다.

기능 분석

↓

기능 구현

↓

테스트

↓

오류 확인

↓

코드 수정

↓

재테스트

↓

Commit

↓

Push

프로젝트 과정에서 Servlet별 기능 수정과 리팩터링 내용을 각각 커밋으로 남겼으며, 기능 구현 이후 코드를 다시 검토하고 리팩터링하는 과정도 Git 기록으로 관리하였다.

20. 프로젝트를 통해 습득한 기술

Java

Java 기반 웹 애플리케이션 개발

객체지향 프로그래밍

DTO 및 DAO 설계

예외 처리

컬렉션 및 객체 관리

Servlet

HTTP 요청 및 응답 처리

Servlet 기반 MVC 구조

세션 관리

Servlet 간 데이터 전달

JSON 응답 처리

Database

Oracle Database

SQL

JDBC

관계형 데이터베이스